TOP 40 HIGH-FREQUENCY & CREATIVE LAB EXAM QUESTIONS

Master Examination Solution Manual • 5 Questions per Core Subject Area

8 Core Modules **40 Complete Solutions** **Exam-Tested Code** **Creative & Frequent Patterns**

Curated question bank reflecting university practical examination trends, lab viva questions, and scenario-based edge-case problems with copy-pasteable, error-free working code.

TOPIC 1 • HTML5 Forms, Complex Tables & Multimedia Page Design (5 Questions)

Q1.1 • University Exam Timetable with Rowspan & Colspan

High Frequency (95%)

Design an Examination Timetable table in HTML having columns (Date, Session, Degree, Subject Code, Subject Name). Merge the cells across 3 days for common core subjects using `rowspan` and merge morning/afternoon session headers using `colspan`.

> **Key Concept:** `rowspan="N"` extends vertical cell height downwards. `colspan="N"` extends horizontal cell width across columns.

```
<!DOCTYPE html>
<html>
<head><title>Semester Exam Timetable</title></head>
<body style="font-family:Arial; padding:20px;">
    <h2 align="center">END SEMESTER PRACTICAL & THEORY EXAMINATION TIMETABLE</h2>
    <table border="2" cellpadding="8" cellspacing="0" width="90%" align="center">
        <tr bgcolor="#cbd5e1" align="center">
            <th rowspan="2">Date</th>
            <th colspan="2">Morning Session (09:30 AM - 12:30 PM)</th>
            <th colspan="2">Afternoon Session (01:30 PM - 04:30 PM)</th>
        </tr>
        <tr bgcolor="#e2e8f0" align="center">
            <th>Code</th><th>Subject Name</th><th>Code</th><th>Subject Name</th>
        </tr>
        <tr align="center">
            <td>10-Oct-2026</td><td>CS301</td><td>Web Technology</td><td>CS302</td><td>Database
Systems</td>
        </tr>
        <tr align="center">
            <td>12-Oct-2026</td>
            <td colspan="2" bgcolor="#dcfce7"><b>COMMON ELECTIVE: Cloud Computing & DevOps</b></td>
            <td>CS304</td><td>Python Programming</td>
        </tr>
        <tr align="center">
            <td>14-Oct-2026</td>
            <td rowspan="2" bgcolor="#fee2e2">CS305</td><td rowspan="2" bgcolor="#fee2e2"><b>Full-Stack Web
Lab (2-Day Session)</b></td>
            <td>CS306</td><td>Cyber Security</td>
        </tr>
        <tr align="center">
            <td>15-Oct-2026</td><td>CS307</td><td>AI & Machine Learning</td>
        </tr>
    </table>
</body>
</html>
```

Q1.2 • Comprehensive Student Admission Form with HTML5 Constraints

High Frequency (90%)

Create an HTML form utilizing all modern HTML5 constraints (`min` , `max` , `step` , `pattern` , `datalist` , `autofocus` , `required`) with image file upload.

```
<!DOCTYPE html>
<html>
<body style="font-family:sans-serif; padding:20px;">
    <form action="process.php" method="POST" enctype="multipart/form-data">
        <fieldset style="width:450px;">
            <legend><b>Student Admission Form</b></legend>
            <label>Roll Number (ABC-1234):</label><br>
            <input type="text" name="roll" pattern="[A-Z]{3}-[0-9]{4}" title="3 Caps, hyphen, 4 digits"
required autofocus><br><br>

            <label>Age (18 - 60):</label><br>
            <input type="number" name="age" min="18" max="60" required><br><br>

            <label>Campus City (Datalist):</label><br>
            <input list="cities" name="city" required>
            <datalist id="cities">
                <option value="Chennai"><option value="Bangalore"><option value="Hyderabad">
            </datalist><br><br>

            <label>Passport Photo:</label><br>
            <input type="file" name="photo" accept="image/png, image/jpeg" required><br><br>

            <button type="submit">Submit Application</button>
        </fieldset>
    </form>
</body>
</html>
```

Q1.3 • Scientific Chemical Equation & Financial Formula Formatter

Creative Scenario

Write an HTML document displaying chemical reaction formulas, quadratic equations, and discounted prices using `<sub>` , `<sup>` , `<del>` , `<ins>` , `<mark>` , and `<abbr>` .

```
<!DOCTYPE html>
<html>
<body style="font-family:sans-serif; padding:20px;">
    <h2>Scientific & Financial Formulas Presentation</h2>
    <ul>
        <li><strong>Sulfuric Acid Reaction:</strong> 2H<sub>2</sub> + O<sub>2</sub> &rarr;
2H<sub>2</sub>O</li>
        <li><strong>Glucose Equation:</strong> 6CO<sub>2</sub> + 6H<sub>2</sub>O &rarr;
C<sub>6</sub>H<sub>12</sub>O<sub>6</sub> + 6O<sub>2</sub></li>
        <li><strong>Algebraic Identity:</strong> (x + y)<sup>3</sup> = x<sup>3</sup> + 3x<sup>2</sup>y +
3xy<sup>2</sup> + y<sup>3</sup></li>
        <li><strong>E-Commerce Discount:</strong> Original Price: <del>Rs. 2,999</del> | Deal Price: <ins
style="color:green;">Rs. 1,499</ins> (<mark>50% OFF</mark>)</li>
        <li><strong>Acronym Definition:</strong> <abbr title="Asynchronous JavaScript and XML">AJAX</abbr>
enables seamless asynchronous web requests.</li>
    </ul>
</body>
</html>
```

Q1.4 • Multi-Frame Portal Navigation Architecture

High Frequency (85%)

Design a 3-part frameset containing a Top Header banner, Left Navigation Menu, and Right Main Target frame with seamless cross-frame anchor linking.

```
<!-- index.html (Master Frameset - No <body> tag!) -->
<!DOCTYPE html>
<html>
<head><title>Portal Frame Architecture</title></head>
<frameset rows="20%, 80%" border="2">
    <frame srcdoc="<h1 align='center' style='color:#1e40af;'>UNIVERSITY ACADEMIC PORTAL</h1>"
name="headerFrame" scrolling="no">
    <frameset cols="25%, 75%">
        <frame srcdoc="<h3>Menu</h3><ul><li><a href='about:blank' target='mainFrame'>Home</a></li><li><a
href='about:blank' target='mainFrame'>Syllabus</a></li></ul>" name="menuFrame">
        <frame srcdoc="<h2>Welcome to Main Workspace</h2><p>Select an option from the left menu.</p>"
name="mainFrame">
    </frameset>
    <noframes><body>Frames not supported.</body></noframes>
</frameset>
</html>
```

Q1.5 • Decimal & Hexadecimal Emoji Badge Generator

Creative Scenario

Write an HTML page that presents a structured catalog of emoji characters using decimal entity codes, hexadecimal entity codes, and direct UTF-8 encoding.

```
<!DOCTYPE html>
<html>
<head><meta charset="UTF-8"><title>Emoji Encoding Catalog</title></head>
<body style="font-family:sans-serif; padding:20px;">
    <h2>HTML5 Unicode Emoji Standards</h2>
    <table border="1" cellpadding="8">
        <tr bgcolor="#e2e8f0"><th>Symbol Name</th><th>Decimal Entity</th><th>Hex Entity</th>
<th>Rendered</th></tr>
        <tr><td>Graduation Cap</td><td>&amp;#127891;</td><td>&amp;#x1F393;</td><td style="font-
size:24px;">&#127891;</td></tr>
        <tr><td>Laptop Computer</td><td>&amp;#128187;</td><td>&amp;#x1F4BB;</td><td style="font-
size:24px;">&#128187;</td></tr>
        <tr><td>Rocket Launch</td><td>&amp;#128640;</td><td>&amp;#x1F680;</td><td style="font-
size:24px;">&#128640;</td></tr>
    </table>
</body>
</html>
```

TOPIC 2 • CSS3 Styling, Specificity Hierarchy & Geometric Drawing (5 Questions)

Q2.1 • Cascade Priority & Specificity Overriding Demonstration

🔥 High Frequency (90%)

Demonstrate how Inline CSS overrides Internal CSS, Internal overrides External, and how `!important` overrides Inline styles in a single HTML file.

```
<!DOCTYPE html>
<html>
<head>
    <style>
        p { color: gray; font-size: 14px; } /* 1. Element Selector (Low) */
        .highlight { color: blue; }         /* 2. Class Selector */
        #special { color: green; }          /* 3. ID Selector */
        .force-red { color: red !important; }/* 4. !important (Supreme) */
    </style>
</head>
<body>
    <p>Default Paragraph (Gray)</p>
    <p class="highlight">Class Selector (Blue)</p>
    <p id="special" class="highlight">ID beats Class (Green)</p>
    <p id="special" style="color: purple;">Inline style beats ID (Purple)</p>
    <p style="color: purple;" class="force-red">!important beats Inline (Red)</p>
</body>
</html>
```

Q2.2 • Pure CSS Geometric Shapes: Circle, Triangle & Shaded Card

✨ Creative Scenario

Draw a Circle (border-radius), an upward Triangle (transparent borders), and a Glassmorphic Gradient Box with 3D box-shadow using zero images.

```
<!DOCTYPE html>
<html>
<head>
<style>
    .shape-container { display:flex; gap:25px; align-items:center; padding:20px; font-family:sans-serif; }
    /* 1. Circle */
    .circle { width:80px; height:80px; background:#16a34a; border-radius:50%; box-shadow: 0 4px 10px
rgba(0,0,0,0.2); }
    /* 2. Triangle using border geometry */
    .triangle { width:0; height:0; border-left:45px solid transparent; border-right:45px solid transparent;
border-bottom:80px solid #ea580c; }
    /* 3. Gradient Box with Shading */
    .shaded-card { width:150px; height:80px; background:linear-gradient(135deg, #2563eb, #9333ea); border-
radius:10px; box-shadow: 6px 10px 20px rgba(37,99,235,0.4); }
</style>
</head>
<body>
    <h3>Pure CSS Drawing Elements</h3>
    <div class="shape-container">
        <div class="circle" title="Circle"></div>
        <div class="triangle" title="Triangle"></div>
        <div class="shaded-card" title="Gradient Box"></div>
    </div>
</body>
</html>
```

Q2.3 • Responsive Navigation Bar with Hover Indicator

🔥 High Frequency (85%)

Create a modern flexbox navigation bar where menu items have smooth hover transitions and active indicator underlines.

```
<!DOCTYPE html>
<html>
<head>
<style>
    .navbar { display:flex; list-style:none; margin:0; padding:12px 20px; background:#1e293b; gap:15px;
border-radius:8px; }
    .navbar li a { color:#f8fafc; text-decoration:none; font-weight:600; padding:8px 12px; border-
radius:4px; transition:all 0.3s; }
    .navbar li a:hover { background:#3b82f6; color:#ffffff; }
</style>
</head>
<body>
    <ul class="navbar">
        <li><a href="#">Dashboard</a></li>
        <li><a href="#">Students</a></li>
        <li><a href="#">Courses</a></li>
        <li><a href="#">Reports</a></li>
    </ul>
</body>
</html>
```

Q2.4 • Interactive CSS Pricing Card with Hover Animation

✨ Creative Scenario

Design an e-commerce subscription card with gradient pricing badge, rounded corners, and smooth `transform: translateY(-5px)` animation on hover.

```
<!DOCTYPE html>
<html>
<head>
<style>
    .card { width:260px; padding:25px; border-radius:12px; background:#ffffff; box-shadow:0 6px 20px
rgba(0,0,0,0.08); border:1px solid #e2e8f0; font-family:sans-serif; text-align:center; transition:transform
0.3s, box-shadow 0.3s; }
    .card:hover { transform:translateY(-8px); box-shadow:0 12px 30px rgba(0,0,0,0.15); }
    .price { font-size:28px; font-weight:bold; color:#2563eb; margin:15px 0; }
    .btn { background:#2563eb; color:white; padding:10px 20px; border:none; border-radius:6px; font-
weight:bold; cursor:pointer; }
</style>
</head>
<body style="background:#f1f5f9; padding:40px;">
    <div class="card">
        <h3>Pro Student Plan</h3>
        <div class="price">Rs. 499<span style="font-size:14px;">/mo</span></div>
        <p>Full access to 32 Lab Experiments & Code Sandbox.</p>
        <button class="btn">Enroll Now</button>
    </div>
</body>
</html>
```

Q2.5 • Multi-Column CSS Form Layout with Accent Colors

High Frequency (80%)

Format a responsive 2-column registration layout using CSS Flexbox with custom `accent-color` for radio buttons and checkboxes.

```
<!DOCTYPE html>
<html>
<head>
<style>
    .form-row { display:flex; gap:15px; margin-bottom:12px; }
    .form-col { flex:1; }
    input[type="text"], select { width:100%; padding:8px; border:1px solid #ccc; border-radius:4px; box-
sizing:border-box; }
    input[type="radio"], input[type="checkbox"] { accent-color: #7c3aed; cursor:pointer; }
</style>
</head>
<body style="font-family:sans-serif; padding:20px;">
    <form style="max-width:450px;">
        <div class="form-row">
            <div class="form-col"><label>First Name:</label><input type="text"></div>
            <div class="form-col"><label>Last Name:</label><input type="text"></div>
        </div>
        <label>Gender:</label><br>
        <input type="radio" name="g" value="M" checked> Male
        <input type="radio" name="g" value="F"> Female
    </form>
</body>
</html>
```

TOPIC 3 • HTML5 Web APIs: Storage, Geolocation & Drag-and-Drop (5 Questions)

Q3.1 • Persistent Visitor Hit Counter using localStorage

High Frequency (95%)

Write an HTML/JavaScript script to maintain a page visit counter using `localStorage` that increments on refresh and provides a "Reset Counter" button.

```
<!DOCTYPE html>
<html>
<body style="font-family:sans-serif; padding:20px;">
    <h2>Page Visit Hit Counter (localStorage)</h2>
    <p style="font-size:18px;">Total Visits: <strong id="hitCount" style="color:#2563eb;">0</strong></p>
    <button onclick="resetHits()" style="padding:8px 14px; background:#dc2626; color:white; border:none;
border-radius:4px; cursor:pointer;">Reset Counter</button>

    <script>
    // Read and increment count on page load
    let visits = Number(localStorage.getItem("pageVisits") || 0) + 1;
    localStorage.setItem("pageVisits", visits);
    document.getElementById("hitCount").textContent = visits;

    function resetHits() {
        localStorage.removeItem("pageVisits");
        document.getElementById("hitCount").textContent = "0 (Reset)";
    }
    </script>
</body>
</html>
```

Q3.2 • Geolocation GPS Coordinates & Google Maps Link Generator

🔥 High Frequency (90%)

Write a script using `navigator.geolocation` to retrieve the user's Latitude and Longitude and generate a clickable Google Maps link.

```
<!DOCTYPE html>
<html>
<body style="font-family:sans-serif; padding:20px;">
    <h2>GPS Geolocation Locator</h2>
    <button onclick="findMe()" style="padding:10px 18px; background:#16a34a; color:white; border:none;
border-radius:4px; cursor:pointer;">📍 Locate My Position</button>
    <div id="mapResult" style="margin-top:15px; font-size:14px;"></div>

    <script>
    function findMe() {
        const out = document.getElementById("mapResult");
        if (!navigator.geolocation) {
            out.innerHTML = "<span style='color:red;'>Geolocation not supported by this browser.</span>";
            return;
        }
        out.innerHTML = "Fetching GPS satellites...";
        navigator.geolocation.getCurrentPosition(
            pos => {
                const lat = pos.coords.latitude;
                const lon = pos.coords.longitude;
                out.innerHTML = `
                    <p><b>Latitude:</b> ${lat}<br><b>Longitude:</b> ${lon}<br><b>Accuracy:</b> Within
${pos.coords.accuracy} meters</p>
                    <a href="https://maps.google.com/?q=${lat},${lon}" target="_blank" style="color:blue;
font-weight:bold;">🗺️ Open in Google Maps</a>
                `;
            },
            err => { out.innerHTML = "<span style='color:red;'>Location permission denied: " + err.message
+ "</span>"; }
        );
    }
    </script>
</body>
</html>
```

Q3.3 • Interactive Drag-and-Drop Task Kanban Board

✨ Creative Scenario

Implement an interactive HTML5 Drag & Drop interface where user can drag student task badges between "Pending" and "Completed" containers.

```
<!DOCTYPE html>
<html>
<head>
<style>
    .board { display:flex; gap:20px; font-family:sans-serif; }
    .col { width:220px; min-height:180px; border:2px dashed #94a3b8; border-radius:8px; padding:12px;
background:#f8fafc; }
    .item { padding:10px; background:#2563eb; color:white; border-radius:6px; margin-bottom:8px;
cursor:grab; font-weight:bold; }
</style>
</head>
<body>
    <h2>Drag & Drop Task Manager</h2>
    <div class="board">
        <div class="col" ondragover="event.preventDefault()" ondrop="drop(event)">
            <h4>⏳ Pending Tasks</h4>
            <div id="task1" class="item" draggable="true" ondragstart="drag(event)">XML Schema DTD Ex</div>
            <div id="task2" class="item" draggable="true" ondragstart="drag(event)">JSON Looping Ex</div>
        </div>
        <div class="col" ondragover="event.preventDefault()" ondrop="drop(event)">
            <h4>✅ Completed Tasks</h4>
        </div>
    </div>
    <script>
    function drag(e) { e.dataTransfer.setData("text", e.target.id); }
    function drop(e) {
        e.preventDefault();
        const id = e.dataTransfer.getData("text");
        const el = document.getElementById(id);
        if (el) e.currentTarget.appendChild(el);
    }
    </script>
</body>
</html>
```

Q3.4 • Difference between localStorage and sessionStorage in Action

High Frequency (85%)

Write a single page that saves data to both `localStorage` and `sessionStorage` and displays the difference when a new tab is opened.

```
<!DOCTYPE html>
<html>
<body style="font-family:sans-serif; padding:20px;">
    <h2>localStorage vs sessionStorage Demo</h2>
    <input type="text" id="userInput" placeholder="Type secret message...">
    <button onclick="saveBoth()">Save to Both Storages</button>
    <div id="display" style="margin-top:15px; padding:15px; background:#f1f5f9; border-radius:6px;"></div>

    <script>
    function saveBoth() {
        const val = document.getElementById("userInput").value;
        localStorage.setItem("localData", val);
        sessionStorage.setItem("sessionData", val);
        render();
    }
    function render() {
        document.getElementById("display").innerHTML = `
            <p><b>localStorage (Persistent):</b> ${localStorage.getItem("localData") || 'Empty'}</p>
            <p><b>sessionStorage (Tab-only):</b> ${sessionStorage.getItem("sessionData") || 'Empty'}</p>
        `;
    }
    render();
    </script>
</body>
</html>
```

Q3.5 • Background Prime Number Calculator using Web Workers

✨ Creative Scenario

Demonstrate non-blocking heavy background mathematical computation using the HTML5 Web Worker API.

```
<!DOCTYPE html>
<html>
<body style="font-family:sans-serif; padding:20px;">
    <h2>Web Worker Non-Blocking Background Task</h2>
    <button onclick="startWorker()">Start Background Counter</button>
    <button onclick="stopWorker()">Stop Worker</button>
    <p>Worker Output: <strong id="workerOut" style="color:green;">Idle</strong></p>

    <script>
    let w;
    function startWorker() {
        if (typeof(Worker) !== "undefined") {
            // Create inline worker via Blob
            const blob = new Blob([`
                let i = 0;
                function timedCount() { i++; postMessage(i); setTimeout(timedCount, 500); }
                timedCount();
            `], { type: 'application/javascript' });

            w = new Worker(URL.createObjectURL(blob));
            w.onmessage = function(e) {
                document.getElementById("workerOut").textContent = "Count: " + e.data;
            };
        }
    }
    function stopWorker() {
        if (w) { w.terminate(); w = undefined; document.getElementById("workerOut").textContent =
"Stopped."; }
    }
    </script>
</body>
</html>
```

TOPIC 4 • Client-Side JavaScript Form Validation & DOM Manipulation (5 Questions)

Q4.1 • Strict Email & 10-Digit Mobile Number Regular Expression Validator

High Frequency (95%)

Write a complete JS validation function to test email matching standard format (e.g. `user@domain.com`) and Indian 10-digit phone number starting with digits 6, 7, 8, or 9.

```
<!DOCTYPE html>
<html>
<head><style>.err { color:red; font-size:12px; display:block; }</style></head>
<body style="font-family:sans-serif; padding:20px;">
    <form onsubmit="return validateRegEx()">
        Email: <input type="text" id="email"><span id="emailErr" class="err"></span><br><br>
        Phone: <input type="text" id="phone"><span id="phoneErr" class="err"></span><br><br>
        <button type="submit">Validate Form</button>
    </form>
    <script>
    function validateRegEx() {
        let valid = true;
        document.getElementById("emailErr").textContent = "";
        document.getElementById("phoneErr").textContent = "";

        const email = document.getElementById("email").value.trim();
        const phone = document.getElementById("phone").value.trim();

        // 1. Email Regex
        const emailPattern = /^[a-zA-Z0-9._%+-]+@[a-zA-Z0-9.-]+\.[a-zA-Z]{2,}$/;
        if (!emailPattern.test(email)) {
            document.getElementById("emailErr").textContent = "Invalid email format (e.g.
name@domain.com)";
            valid = false;
        }

        // 2. 10-Digit Phone Regex starting with 6, 7, 8, 9
        const phonePattern = /^[6-9][0-9]{9}$/;
        if (!phonePattern.test(phone)) {
            document.getElementById("phoneErr").textContent = "Must be a valid 10-digit mobile number
starting with 6-9";
            valid = false;
        }

        return valid;
    }
    </script>
</body>
</html>
```

Q4.2 • Password Strength Meter & Confirmation Matcher

High Frequency (90%)

Write a real-time password strength checker that dynamically updates a color bar based on length, uppercase, numbers, and special characters, and validates password confirmation matching.

```
<!DOCTYPE html>
<html>
<body style="font-family:sans-serif; padding:20px;">
    <label>Password:</label><br>
    <input type="password" id="pwd" oninput="checkStrength()">
    <span id="strText" style="font-weight:bold; font-size:12px;">Strength: None</span><br><br>

    <label>Confirm Password:</label><br>
    <input type="password" id="cpwd"><span id="matchErr" style="color:red; font-size:12px;"></span><br><br>

    <button onclick="validateMatch()">Verify Match</button>

    <script>
    function checkStrength() {
        const val = document.getElementById("pwd").value;
        let score = 0;
        if (val.length >= 6) score++;
        if (/[A-Z]/.test(val)) score++;
        if (/[0-9]/.test(val)) score++;
        if (/[^A-Za-z0-9]/.test(val)) score++;

        const txt = document.getElementById("strText");
        if (score <= 1) { txt.textContent = "Weak"; txt.style.color = "red"; }
        else if (score <= 3) { txt.textContent = "Medium"; txt.style.color = "orange"; }
        else { txt.textContent = "Strong"; txt.style.color = "green"; }
    }
    function validateMatch() {
        const p1 = document.getElementById("pwd").value;
        const p2 = document.getElementById("cpwd").value;
        const err = document.getElementById("matchErr");
        if (p1 !== p2) err.textContent = " Passwords do not match!";
        else { err.textContent = " Passwords Match!"; err.style.color = "green"; }
    }
    </script>
</body>
</html>
```

Q4.3 • Dynamic Course Dropdown populated based on Degree Selection

✨ Creative Scenario

Implement cascading dropdown menus where selecting Degree (UG vs PG) dynamically updates the available Course options in the second dropdown.

```
<!DOCTYPE html>
<html>
<body style="font-family:sans-serif; padding:20px;">
    <h2>Cascading Degree & Course Selector</h2>
    <label>Select Degree Level:</label><br>
    <select id="degreeSelect" onchange="updateCourses()">
        <option value="">-- Select Level --</option>
        <option value="UG">Undergraduate (UG)</option>
        <option value="PG">Postgraduate (PG)</option>
    </select><br><br>

    <label>Available Branch / Course:</label><br>
    <select id="courseSelect">
        <option value="">-- First select degree level --</option>
    </select>

    <script>
    const courses = {
        UG: ["B.Sc Computer Science", "BCA", "B.Tech Information Technology"],
        PG: ["MCA", "M.Sc Computer Science", "M.Tech AI & Data Science"]
    };
    function updateCourses() {
        const deg = document.getElementById("degreeSelect").value;
        const cSelect = document.getElementById("courseSelect");
        cSelect.innerHTML = "";
        if (!deg || !courses[deg]) {
            cSelect.innerHTML = "<option>-- First select degree level --</option>";
            return;
        }
        courses[deg].forEach(c => {
            let opt = document.createElement("option");
            opt.value = c;
            opt.textContent = c;
            cSelect.appendChild(opt);
        });
    }
    </script>
</body>
</html>
```

Q4.4 • Mandatory Radio & Minimum Checkbox Selection Validator

High Frequency (85%)

Write JavaScript code verifying that Gender radio is chosen and at least 2 Technical Skills checkboxes are ticked before allowing form submission.

```
<!DOCTYPE html>
<html>
<body style="font-family:sans-serif; padding:20px;">
    <form onsubmit="return validateSelection(event)">
        <label><b>Gender:</b></label><br>
        <input type="radio" name="gender" value="Male"> Male
        <input type="radio" name="gender" value="Female"> Female<br><br>

        <label><b>Technical Skills (Select at least 2):</b></label><br>
        <input type="checkbox" name="skill" value="HTML"> HTML
        <input type="checkbox" name="skill" value="JS"> JavaScript
        <input type="checkbox" name="skill" value="PHP"> PHP
        <input type="checkbox" name="skill" value="MySQL"> MySQL<br><br>

        <button type="submit">Validate & Enroll</button>
    </form>
    <script>
    function validateSelection(e) {
        const gender = document.querySelector('input[name="gender"]:checked');
        if (!gender) {
            alert("Error: Please select your Gender.");
            e.preventDefault();
            return false;
        }
        const skills = document.querySelectorAll('input[name="skill"]:checked');
        if (skills.length < 2) {
            alert("Error: Please select at least TWO technical skills.");
            e.preventDefault();
            return false;
        }
        alert("Success: All radio and checkbox constraints verified!");
        return true;
    }
    </script>
</body>
</html>
```

Q4.5 • Real-Time Live Image Preview Reader using FileReader API

✨ Creative Scenario

Create an avatar photo upload control that verifies the file size (< 2MB) and renders an instant circular thumbnail preview before uploading.

```
<!DOCTYPE html>
<html>
<body style="font-family:sans-serif; padding:20px;">
    <h3>Instant Photo Preview & Size Check</h3>
    <div style="display:flex; align-items:center; gap:20px;">
        <img id="avatarThumb" src="about:blank" style="width:70px; height:70px; border-radius:50%;
border:2px solid #ccc; background:#e2e8f0;" alt="Preview">
        <input type="file" id="photoInput" accept="image/*" onchange="previewAvatar(event)">
    </div>
    <p id="fileErr" style="color:red; font-size:12px;"></p>

    <script>
    function previewAvatar(e) {
        const file = e.target.files[0];
        const err = document.getElementById("fileErr");
        err.textContent = "";
        if (!file) return;

        if (file.size > 2 * 1024 * 1024) { // 2MB Limit
            err.textContent = "File exceeds 2MB size limit!";
            e.target.value = "";
            return;
        }

        const reader = new FileReader();
        reader.onload = function(evt) {
            document.getElementById("avatarThumb").src = evt.target.result;
        };
        reader.readAsDataURL(file);
    }
    </script>
</body>
</html>
```

TOPIC 5 • Full-Stack PHP & MySQL Database Programming (5 Questions)

Q5.1 • Secure Prepared Statement Insert with Bcrypt Password Hash

High Frequency (95%)

Write PHP script connecting to MySQL database (`student_db`), hashing user password with `password_hash(..., PASSWORD_DEFAULT)` , and inserting data via prepared statements.

```php
<?php
// save_student.php
$servername = "localhost";
$username   = "root";
$password   = "";
$database   = "student_db";

$conn = new mysqli($servername, $username, $password, $database);
if ($conn->connect_error) {
    die("Database Connection Failed: " . $conn->connect_error);
}

if ($_SERVER["REQUEST_METHOD"] == "POST") {
    $name     = trim($_POST["name"]);
    $email    = trim($_POST["email"]);
    $raw_pass = $_POST["password"];
    $course   = trim($_POST["course"]);

    // Secure Hashing
    $hashed_pass = password_hash($raw_pass, PASSWORD_DEFAULT);

    // Prepared Statement
    $stmt = $conn->prepare("INSERT INTO students (name, email, password, course) VALUES (?, ?, ?, ?)");
    $stmt->bind_param("ssss", $name, $email, $hashed_pass, $course);

    if ($stmt->execute()) {
        echo "<h3 style='color:green;'>Registration Complete! Student ID: " . $stmt->insert_id . "</h3>";
    } else {
        echo "Error: " . $stmt->error;
    }
    $stmt->close();
}
$conn->close();
?>
```

Q5.2 • Searchable Live MySQL Table Display with PHP

High Frequency (90%)

Write a PHP script to fetch all student records from MySQL and render an HTML table with keyword search filter on Name and Course.

```
<?php
$conn = new mysqli("localhost", "root", "", "student_db");
$search = trim($_GET["search"] ?? "");

$sql = "SELECT id, name, email, course, created_at FROM students WHERE 1=1";
if (!empty($search)) {
    $searchEsc = $conn->real_escape_string($search);
    $sql .= " AND (name LIKE '%$searchEsc%' OR course LIKE '%$searchEsc%')";
}
$result = $conn->query($sql);
?>
<form method="GET">
    <input type="search" name="search" value="<?php echo htmlspecialchars($search); ?>" placeholder="Search
name/course...">
    <button type="submit">Search</button>
</form>
<table border="1" cellpadding="8" style="margin-top:10px; width:100%;">
    <tr bgcolor="#e2e8f0"><th>ID</th><th>Name</th><th>Email</th><th>Course</th></tr>
    <?php while ($row = $result->fetch_assoc()): ?>
        <tr>
            <td><?php echo $row['id']; ?></td>
            <td><b><?php echo htmlspecialchars($row['name']); ?></b></td>
            <td><?php echo htmlspecialchars($row['email']); ?></td>
            <td><?php echo htmlspecialchars($row['course']); ?></td>
        </tr>
    <?php endwhile; ?>
</table>
<?php $conn->close(); ?>
```

Q5.3 • Edit & Update Record in MySQL by ID

✨ Creative Scenario

Implement an edit form that pre-populates existing student data from MySQL using `id` and saves updates via `UPDATE students SET ...`.

```
<?php
$conn = new mysqli("localhost", "root", "", "student_db");
$id = intval($_GET["id"] ?? 1);

if ($_SERVER["REQUEST_METHOD"] == "POST") {
    $name   = $_POST["name"];
    $course = $_POST["course"];
    $stmt = $conn->prepare("UPDATE students SET name = ?, course = ? WHERE id = ?");
    $stmt->bind_param("ssi", $name, $course, $id);
    $stmt->execute();
    echo "<p style='color:green;'>Record Updated Successfully!</p>";
}

$res = $conn->query("SELECT * FROM students WHERE id = $id");
$stu = $res->fetch_assoc();
?>
<form method="POST">
    <h3>Edit Student ID: <?php echo $id; ?></h3>
    Name: <input type="text" name="name" value="<?php echo htmlspecialchars($stu['name']); ?>"><br><br>
    Course: <input type="text" name="course" value="<?php echo htmlspecialchars($stu['course']); ?>"><br>
<br>
    <button type="submit">Save Changes</button>
</form>
<?php $conn->close(); ?>
```

Q5.4 • Delete Student Record with JavaScript Confirmation

🔥 High Frequency (85%)

Write PHP script to delete student record securely using prepared statement with client-side `confirm()` dialog.

```
<?php
$conn = new mysqli("localhost", "root", "", "student_db");

if (isset($_GET["del_id"])) {
    $del_id = intval($_GET["del_id"]);
    $stmt = $conn->prepare("DELETE FROM students WHERE id = ?");
    $stmt->bind_param("i", $del_id);
    $stmt->execute();
    echo "<p style='color:red;'>Student ID $del_id deleted from database!</p>";
}
?>
<a href="delete.php?del_id=5" onclick="return confirm('Are you sure you want to delete this student?');"
style="color:red; font-weight:bold;">🗑️ Delete Student 5</a>
<?php $conn->close(); ?>
```

Q5.5 • Export MySQL Records directly to CSV File

✨ Creative Scenario

Write a PHP script that triggers an instant downloadable `.csv` spreadsheet containing all student database records.

```php
<?php
// export_csv.php
$conn = new mysqli("localhost", "root", "", "student_db");

header("Content-Type: text/csv; charset=utf-8");
header("Content-Disposition: attachment; filename=students_export_" . date("Ymd") . ".csv");

$output = fopen("php://output", "w");
fputcsv($output, ["ID", "Name", "Email", "Course", "Registration Date"]);

$result = $conn->query("SELECT id, name, email, course, created_at FROM students");
while ($row = $result->fetch_assoc()) {
    fputcsv($output, $row);
}
fclose($output);
$conn->close();
exit();
?>
```

TOPIC 6 • XML Documents, DTD & XML Schema (XSD) Validation (5 Questions)

Q6.1 • Complete XML Document with External DTD

High Frequency (95%)

Create an external DTD (`students.dtd`) defining element hierarchy with mandatory attributes, and write the valid conforming XML document.

```
<!-- students.dtd -->
<!ELEMENT students (student+)>
<!ELEMENT student (id, name, course, email)>
<!ATTLIST student rollNo CDATA #REQUIRED status (active|graduated) "active">
<!ELEMENT id (#PCDATA)>
<!ELEMENT name (#PCDATA)>
<!ELEMENT course (#PCDATA)>
<!ELEMENT email (#PCDATA)>

<!-- students.xml -->
<?xml version="1.0" encoding="UTF-8"?>
<!DOCTYPE students SYSTEM "students.dtd">
<students>
    <student rollNo="CSB-2026" status="active">
        <id>101</id>
        <name>Alex Johnson</name>
        <course>B.Sc Computer Science</course>
        <email>alex@example.com</email>
    </student>
</students>
```

Q6.2 • XSD with Regex Pattern, Min/Max Integer Range & Enumeration

High Frequency (95%)

Write an XML Schema (`students.xsd`) enforcing (1) Roll Number format `[A-Z]{3}-[0-9]{4}` , (2) Age range 18–60, and (3) Gender enumeration (Male|Female|Other).

```
<?xml version="1.0" encoding="UTF-8"?>
<xs:schema xmlns:xs="http://www.w3.org/2001/XMLSchema">
    <xs:element name="student">
        <xs:complexType>
            <xs:sequence>
                <!-- 1. Regex Pattern -->
                <xs:element name="rollNo">
                    <xs:simpleType>
                        <xs:restriction base="xs:string">
                            <xs:pattern value="[A-Z]{3}-[0-9]{4}"/>
                        </xs:restriction>
                    </xs:simpleType>
                </xs:element>

                <!-- 2. Integer Range -->
                <xs:element name="age">
                    <xs:simpleType>
                        <xs:restriction base="xs:integer">
                            <xs:minInclusive value="18"/>
                            <xs:maxInclusive value="60"/>
                        </xs:restriction>
                    </xs:simpleType>
                </xs:element>

                <!-- 3. Enumeration -->
                <xs:element name="gender">
                    <xs:simpleType>
                        <xs:restriction base="xs:string">
                            <xs:enumeration value="Male"/>
                            <xs:enumeration value="Female"/>
                            <xs:enumeration value="Other"/>
                        </xs:restriction>
                    </xs:simpleType>
                </xs:element>
            </xs:sequence>
        </xs:complexType>
    </xs:element>
</xs:schema>
```

Q6.3 • Validate XML against XSD Schema in PHP

✨ Creative Scenario

Write a PHP script that loads an XML document, executes `$dom->schemaValidate("students.xsd")`, and prints any schema error messages with line numbers.

```php
<?php
libxml_use_internal_errors(true);

$dom = new DOMDocument();
$dom->load("students.xml");

if (@$dom->schemaValidate("students.xsd")) {
    echo "<h3 style='color:green;'>✅ XML conforms 100% to XSD Schema!</h3>";
} else {
    echo "<h3 style='color:red;'>❌ XSD Schema Validation Errors:</h3><ul>";
    foreach (libxml_get_errors() as $err) {
        echo "<li><b>Line {$err->line}:</b> {$err->message}</li>";
    }
    echo "</ul>";
}
libxml_clear_errors();
?>
```

Q6.4 • Client-side XML Parsing to HTML Table via DOMParser

🔥 High Frequency (90%)

Write JavaScript code using `DOMParser` to parse an XML text payload and dynamically construct an HTML table.

```html
<!DOCTYPE html>
<html>
<body style="font-family:sans-serif; padding:20px;">
    <table border="1" id="xmlTbl" cellpadding="8" style="width:100%; border-collapse:collapse;">
        <tr bgcolor="#e2e8f0"><th>ID</th><th>Student Name</th><th>Course</th></tr>
    </table>
    <script>
    const xmlText = `<students>
        <student><id>101</id><name>Alex</name><course>BSc CS</course></student>
        <student><id>102</id><name>Sophia</name><course>MCA</course></student>
    </students>`;

    const parser = new DOMParser();
    const xmlDoc = parser.parseFromString(xmlText, "text/xml");
    const students = xmlDoc.getElementsByTagName("student");
    const table = document.getElementById("xmlTbl");

    for (let i = 0; i < students.length; i++) {
        let id = students[i].getElementsByTagName("id")[0].textContent;
        let name = students[i].getElementsByTagName("name")[0].textContent;
        let course = students[i].getElementsByTagName("course")[0].textContent;

        let row = table.insertRow();
        row.innerHTML = `<td>${id}</td><td><b>${name}</b></td><td>${course}</td>`;
    }
    </script>
</body>
</html>
```

Q6.5 • Two-Way XML Form Sync (Form → XML & XML → Form)

✨ Creative Scenario

Implement an HTML form that can both save inputs to XML and prefill form fields from an existing XML document.

```
<!DOCTYPE html>
<html>
<body style="font-family:sans-serif; padding:20px;">
    <form id="syncForm">
        Name: <input id="name" value="Alex"><br><br>
        Course: <input id="course" value="BSc CS"><br><br>
        <button type="button" onclick="loadXml()">1. Prefill from XML</button>
        <button type="button" onclick="exportXml()">2. Export to XML File</button>
    </form>
    <script>
    function loadXml() {
        const sampleXml = "<student><name>Sophia Miller</name><course>MCA</course></student>";
        const doc = new DOMParser().parseFromString(sampleXml, "text/xml");
        document.getElementById("name").value = doc.querySelector("name").textContent;
        document.getElementById("course").value = doc.querySelector("course").textContent;
    }
    function exportXml() {
        const n = document.getElementById("name").value;
        const c = document.getElementById("course").value;
        const xml = `<?xml version="1.0"?><student><name>${n}</name><course>${c}</course></student>`;
        const a = document.createElement("a");
        a.href = URL.createObjectURL(new Blob([xml], {type:"application/xml"}));
        a.download = "student.xml";
        a.click();
    }
    </script>
</body>
</html>
```

TOPIC 7 • Server-Side XML Processing in PHP (5 Questions)

Q7.1 • XML String Parsing with simplexml_load_string()

High Frequency (95%)

Write a PHP script parsing an XML string variable and outputting element properties and attribute values.

```
<?php
$xmlString = <<<XML
<?xml version="1.0" encoding="UTF-8"?>
<student id="101" status="active">
    <name>Alex Johnson</name>
    <course>B.Sc Computer Science</course>
    <city>Chennai</city>
</student>
XML;

$xml = simplexml_load_string($xmlString) or die("Error parsing XML string");
echo "<h3>Parsed Student Record</h3>";
echo "ID Attribute: " . $xml["id"] . "<br>";
echo "Student Name: " . $xml->name . "<br>";
echo "Course: " . $xml->course . "<br>";
echo "City: " . $xml->city . "<br>";
?>
```

Q7.2 • XML File Parsing and Looping with foreach

High Frequency (95%)

Write PHP script loading `students.xml` from disk and using `foreach ($xml->student as $s)` to construct a complete styled table.

```
<?php
$xml = simplexml_load_file("students.xml") or die("Cannot load students.xml");
?>
<h2>Students List (Parsed from XML via PHP)</h2>
<table border="1" cellpadding="8" style="border-collapse:collapse; width:100%; font-family:sans-serif;">
    <tr bgcolor="#e2e8f0"><th>ID</th><th>Roll No</th><th>Name</th><th>Course</th><th>Email</th></tr>
    <?php foreach ($xml->student as $s): ?>
        <tr>
            <td><?php echo $s->id; ?></td>
            <td><span style="background:#e0f2fe; padding:2px 6px; border-radius:4px;"><?php echo $s-
>rollNo; ?></span></td>
            <td><b><?php echo $s->name; ?></b></td>
            <td><?php echo $s->course; ?></td>
            <td><?php echo $s->email; ?></td>
        </tr>
    <?php endforeach; ?>
</table>
```

Q7.3 • Append New Node to Existing XML File using DOMDocument

✨ Creative Scenario

Write PHP script that safely appends a new student record into `students.xml` maintaining proper formatting indentation.

```
<?php
$xml_file = "students.xml";
$dom = new DOMDocument("1.0", "UTF-8");
$dom->preserveWhiteSpace = false;
$dom->formatOutput = true;
$dom->load($xml_file);

// Create new student element
$student = $dom->createElement("student");
$student->appendChild($dom->createElement("id", "3"));
$student->appendChild($dom->createElement("name", "David Warner"));
$student->appendChild($dom->createElement("course", "BCA"));

$dom->documentElement->appendChild($student);
$dom->save($xml_file);
echo "<h3 style='color:green;'>Appended new student node to $xml_file!</h3>";
?>
```

Q7.4 • Filter & Search XML Elements in PHP

🔥 High Frequency (85%)

Write PHP program to search and display only students where `course == 'MCA'` and `age > 21` from XML file.

```
<?php
$xml = simplexml_load_file("students.xml");
echo "<h3>Filtered MCA Students (Age > 21)</h3><ul>";
foreach ($xml->student as $s) {
    if ((string)$s->course === "MCA" && (int)$s->age > 21) {
        echo "<li><b>{$s->name}</b> (Age: {$s->age}) - Email: {$s->email}</li>";
    }
}
echo "</ul>";
?>
```

Q7.5 • Convert XML File directly to JSON Format in PHP

✨ Creative Scenario

Write PHP script reading `students.xml` and converting it to formatted JSON text using `json_encode($xml)`.

```
<?php
$xml = simplexml_load_file("students.xml");
$jsonData = json_encode($xml, JSON_PRETTY_PRINT);

header("Content-Type: application/json");
echo $jsonData;
?>
```

TOPIC 8 • JSON Masterclass, parse, stringify, Looping & File Sync (5 Questions)

Q8.1 • Parse JSON String to Array & Access Values via 5 Loop Types

High Frequency (95%)

Write JS program demonstrating `JSON.parse()` and iterating values via `for...in`, `for...of`, `forEach()`, `map()`, and `filter()`.

```
<script>
const jsonStr = `[
    { "id": 1, "name": "Alex", "course": "BSc CS", "mark": 85 },
    { "id": 2, "name": "Sophia", "course": "MCA", "mark": 92 },
    { "id": 3, "name": "David", "course": "BCA", "mark": 78 }
]`;

const list = JSON.parse(jsonStr);

// 1. for...of loop
for (let s of list) { console.log("Student:", s.name); }

// 2. forEach() method
list.forEach((s, i) => console.log(`[${i}] ${s.name} - ${s.course}`));

// 3. map() transformation
const names = list.map(s => s.name);
console.log("All Names:", names); // ["Alex", "Sophia", "David"]

// 4. filter() query
const highScorers = list.filter(s => s.mark >= 80);
console.log("Mark >= 80:", highScorers.map(s => s.name));
</script>
```

Q8.2 • JSON Date Reviver Function & Function Stringify Restore

High Frequency (95%)

Demonstrate how to restore Date objects during `JSON.parse()` using a Reviver function and how to serialize functions with `.toString()` and `eval()`.

```
<script>
// 1. Date Reviver
const jsonPayload = '{"name": "Emma", "birthDate": "2005-04-12T00:00:00.000Z"}';
const person = JSON.parse(jsonPayload, function(key, value) {
    if (key === "birthDate") return new Date(value); // Restores real Date object
    return value;
});
console.log("Birth Year:", person.birthDate.getFullYear()); // 2005

// 2. Function Stringify & Restore
const calc = {
    title: "Calculator",
    add: function(a, b) { return a + b; }.toString()
};
const serialized = JSON.stringify(calc);
const restored = JSON.parse(serialized);
restored.add = eval("(" + restored.add + ")");
console.log("Restored Add 10+20:", restored.add(10, 20)); // 30
</script>
```

Q8.3 • Send HTML Form Data to JSON File via Fetch API

🔥 High Frequency (90%)

Create HTML form that collects user details, serializes to JSON string, and sends via Fetch POST to PHP backend to append into `students.json`.

```
<!-- Client HTML & JS -->
<form id="studentForm">
    Roll: <input name="roll" value="MCA-2026"><br>
    Name: <input name="name" value="Sophia"><br>
    <button type="button" onclick="submitJson()">Save to JSON File</button>
</form>
<script>
async function submitJson() {
    const form = document.getElementById("studentForm");
    const obj = Object.fromEntries(new FormData(form).entries());
    const res = await fetch("save.php", {
        method: "POST",
        headers: { "Content-Type": "application/json" },
        body: JSON.stringify(obj)
    });
    const data = await res.json();
    alert("Response: " + data.message);
}
</script>

<!-- Backend save.php -->
<?php
$input = json_decode(file_get_contents("php://input"), true);
$file = "students.json";
$list = file_exists($file) ? json_decode(file_get_contents($file), true) : [];
$list[] = $input;
file_put_contents($file, json_encode($list, JSON_PRETTY_PRINT));
echo json_encode(["status" => "success", "message" => "Appended to students.json"]);
?>
```

Q8.4 • Store HTML Form Data as a Separate Individual JSON File

✨ Creative Scenario

Implement client-side direct download of HTML form data as an individual `student_{rollNo}.json` file using JavaScript `Blob` and `URL.createObjectURL()`.

```
<!DOCTYPE html>
<html>
<body style="font-family:sans-serif; padding:20px;">
    <h2>Store Form as Separate JSON File</h2>
    <form id="regForm">
        Roll No: <input type="text" name="rollNo" value="CSB-2026"><br><br>
        Name: <input type="text" name="name" value="Alex Johnson"><br><br>
        Course: <input type="text" name="course" value="B.Sc Computer Science"><br><br>
        <button type="button" onclick="downloadSeparateFile()">📩 Download .json File</button>
    </form>
    <script>
    function downloadSeparateFile() {
        const form = document.getElementById("regForm");
        const obj = Object.fromEntries(new FormData(form).entries());
        obj.savedAt = new Date().toLocaleString();

        const jsonString = JSON.stringify(obj, null, 2);
        const blob = new Blob([jsonString], { type: "application/json" });
        const a = document.createElement("a");
        a.href = URL.createObjectURL(blob);
        a.download = `student_${obj.rollNo}.json`;
        a.click();
        URL.revokeObjectURL(a.href);
    }
    </script>
</body>
</html>
```

Q8.5 • Load & Populate JSON File Data Back into HTML Form Fields

🔥 High Frequency (90%)

Write JavaScript function that reads a JSON object and automatically maps its properties back into matching HTML form text inputs, select dropdowns, and radio buttons.

```
<!DOCTYPE html>
<html>
<body style="font-family:sans-serif; padding:20px;">
    <h2>JSON to Form Auto-Fill</h2>
    <form id="editForm">
        Roll: <input id="roll" name="roll"><br><br>
        Name: <input id="name" name="name"><br><br>
        Course: <select id="course" name="course">
            <option value="BSc CS">BSc CS</option>
            <option value="MCA">MCA</option>
        </select><br><br>
        Gender:
        <input type="radio" name="gender" value="Male"> Male
        <input type="radio" name="gender" value="Female"> Female<br><br>
        <button type="button" onclick="populateForm()">🔄 Populate from JSON</button>
    </form>
    <script>
    function populateForm() {
        const jsonData = {
            "roll": "MCA-2026",
            "name": "Sophia Miller",
            "course": "MCA",
            "gender": "Female"
        };
        const form = document.getElementById("editForm");
        form.elements["roll"].value = jsonData.roll;
        form.elements["name"].value = jsonData.name;
        form.elements["course"].value = jsonData.course;

        // Radio selection
        const radio = form.querySelector(`input[name="gender"][value="${jsonData.gender}"]`);
        if (radio) radio.checked = true;

        alert("Data populated into form fields successfully!");
    }
    </script>
</body>
</html>
```